

AI / ML INTERN TAKE-HOME ASSESSMENT

Adaptive Document Preparation System

Estimated Completion Time: 5 days

Submission Deadline: As communicated by your recruiter

1. Problem Statement

Your task is to build an **Adaptive Document Preparation System** — a backend-driven application that:

- Ingests a multi-section PDF document (~50 pages, 10 sections).
- Allows a user to select one or more sections they want to prepare for.
- Generates a set of Multiple Choice Questions (MCQs) derived from those sections using an LLM.
- Collects user answers and scores them, providing correct answers and clarifications for every wrong answer.
- Persists a knowledge base (KB) of each preparation session — what was asked, what the user got right/wrong.
- On subsequent runs over the same sections, adapts the question set by focusing on the user's historical weak areas (topics with consistent wrong answers) while avoiding excessive repetition of questions already mastered.

Key insight from the spec: The system must distinguish between a first-time prep run and a returning run. On returning runs, the history context (mistakes + question drift) should influence what new questions are generated — **this is the core adaptive intelligence requirement.**

2. Materials Provided

The following will be shared with you alongside this document:

1.  **SLATEFALL_DOSSIER.pdf** — A ~50-page PDF (machine-readable text only, no scanned page or image). This is your primary ingestion corpus.
2. **This assessment document.**

3. What You Build

3.1 Core System — The Prep Feature

Implement the following workflow as your primary deliverable. The flow below is derived directly from the system specification:

PREP FLOW

INPUT : User supplies section IDs to study (e.g., sections 3, 7)

STEP 1 : System checks the KB for any prior prep history for those sections.

STEP 2 : Generate N MCQs using an LLM (N is configurable; default = 5 per section).

STEP 3 : Present questions to the user and collect answers.

STEP 4 : Score the session.

- For each correct answer : mark as correct.

- For each wrong answer : show the correct answer + a concise clarification.

STEP 5 : Persist the result (questions asked, user answers, scores etc.) to the KB.

3.2 Knowledge Base Design

Your KB must support the following query patterns:

- Given a set of section IDs, retrieve all prior prep sessions involving those sections.
- Given a session, retrieve the individual question-level results (which were wrong/right).
- Identify topics/questions that have been answered incorrectly across multiple sessions (for adaptive weighting).
- Retrieve a snapshot of the KB state at the end of any given session (required for the evaluation scenario in Section 7).

A **KB snapshot** is a human-readable export of the top-5 most recent session records at the moment an iteration completes. It should show enough detail for a reviewer to verify that history is being stored correctly and that adaptive prompting is grounded in real data.

You may use any storage backend: SQLite, PostgreSQL, a vector store, plain JSON files, or a hybrid. **Document your schema and access patterns in the README.**

3.3 LLM Integration

- Use a free or local LLM (e.g., Ollama, Groq, Gemini, HuggingFace) to extract and understand selected PDF sections.
- Generate MCQs with answer choices, correct answers, and explanations.
- Incorporate historical context from prior sessions into prompts to improve question relevance and continuity.
- **Document what you chose and why**

3.4 API Layer

Expose the core system functionality through REST APIs. Implement all necessary endpoints required to support the system features and integrations.

4. Technology Stack & AI Tool

You may use any technology stack and AI tool you are comfortable with. There are no restrictions on language, framework, or tooling, **ensuring:**

- The system runs locally or against free-tier cloud services.
- Setup instructions in the README allow a reviewer to run the project from scratch in under 10 minutes.

Suggested (*not required*) starting points:

Component	Suggestions
Backend	FastAPI (Python), Flask, Express (Node.js), Go Gin
LLM	Ollama + LLaMA3 / Mistral; Groq free tier; Gemini Flash
PDF Parsing	PyMuPDF, pdfplumber, LangChain PDF loaders
Vector Store	ChromaDB, FAISS, PGVector, Qdrant (local mode)
Relational DB	SQLite (simplest), PostgreSQL
NoSQL	MongoDB
Orchestration	LangChain, LangGraph, LlamaIndex, raw API calls
UI (optional)	Streamlit, Gradio, React, plain HTML/JS

Document your stack choices and the reasoning behind them in the README. Reviewers will give credit for deliberate, well-justified choices — even if they differ from the suggestions above.

5. Optional Enhancements

The following are not required for a passing submission but carry additional credit when the core implementation is sound:

Enhancement	Description
Containerization	Docker + docker-compose so the full stack runs with a single command.
Minimal UI	A simple interface (Streamlit, Gradio, or web) to interactively run prep sessions and view history.
Test Coverage	Unit and/or integration tests for the core prep flow, KB read/write, and MCQ generation prompts.
Error Handling	Graceful handling of LLM failures, malformed PDFs, invalid section IDs, and concurrent session conflicts.
Logging	Structured logs (timestamps, session IDs, LLM token usage, KB ops) that a reviewer can inspect post-run.

6. Evaluation Scenario & Submission

Submit a public Git repository (GitHub or GitLab) with a CLI-runnable system.

Example evaluation scenarios:

[for evaluation's sake you need to export corresponding outputs as described below.]

Scenario A — run a cold-start prep over any two sections.

Scenario B — three consecutive iterations (save outputs to the folders below):

Iter 1 : sections 5, 8 Iter 2 : sections 6, 8, 9 Iter 3 : section 8

Save scenario B in `repo\_root/outputs`

your-project/

outputs/scenario_b_iter1/ questions_iter1.json + kb_snapshot_iter1.json

outputs/scenario_b_iter2/ questions_iter2.json + kb_snapshot_iter2.json

outputs/scenario_b_iter3/ questions_iter3.json + kb_snapshot_iter3.json

Note: You do not need to have a real human complete the MCQs. **You may simulate user answers** (e.g., random, all-correct, or partially-wrong) for the purpose of the submission outputs. What matters is that the flow executes correctly end-to-end.

6.1 README Requirements

Your README must include:

- Project overview, prerequisites, and setup steps.
- Commands to run each evaluation scenario.
- Stack choices with reasoning, and any known limitations or assumptions about the PDF or LLM behaviour.

6.3 Evaluation Criteria Summary

Reviewers will assess your submission across the following dimensions:

Dimension	Weight	What Reviewers Look For
Functional Correctness	30%	Does the full prep flow execute correctly for both scenarios?
Knowledge Base Design	20%	Is the schema well-thought-out and correctly queryable?
Retrieval / Adaptation	20%	Does iteration 2+ genuinely use history to shape new questions?
Code Quality	10%	Readability, structure, modularity, testing, logging, error handling
Documentation	15%	README clarity (architecture, setup, run commands, etc.), inline comments, design justifications
Optional Enhancements	5%	Bonus credit on a solid core

7. Important Notes

1. **No paid APIs required:** If your chosen LLM requires a paid key, switch to a free alternative before submitting. Submissions that require paid services will be run against a free-tier substitute — ensure your code handles this gracefully.
2. **Simulated user answers are acceptable:** You do not need to recruit a human tester. Simulate a realistic mix of correct and incorrect responses for Scenario B to demonstrate adaptive behavior.
3. **Section numbering in the test PDF:** If the provided PDF uses different section numbering than sections 1–10, map accordingly and document your mapping.
4. **LLM outputs non-determinism:** MCQ generation will vary between runs due to temperature. This is expected. What matters is structural correctness (4 choices, one answer, one explanation), not identical output across runs.
5. **Development Focus:** CLI implementation supporting Scenario B (history-adaptive) functionality first. That is the core differentiator. Optional enhancements should only be attempted after the core flow is solid.

Good luck — we look forward to reviewing your work.
